Synapse AI Tutor MVP

Review-Ready End-to-End Walkthrough

React 19 + Vite Frontend | FastAPI Backend | RAG + Knowledge Graph + Student Memory

1. System Overview

Synapse is an adaptive AI tutoring MVP built with a React + Vite frontend and a FastAPI backend. The system combines retrieval-augmented generation, a knowledge graph, persistent student memory, assessment-based progress tracking, and voice features to create personalized tutoring responses.

```
User -> React UI -> Axios / SSE -> FastAPI /api/v1 routes
     -> JWT auth dependency
     -> RAG / Knowledge Graph / Memory / Progress modules
     -> LLM client builds adaptive response
     -> Streamed answer returns to UI
     -> Student memory and progress JSON files are updated
```

The active MVP persistence layer is JSON-file storage under synapse_ai_tutor/data/. PostgreSQL models exist and describe a future production schema, but most runtime flows currently use JSON repositories and direct JSON helpers.

2. Folder Structure

- frontend/: React 19 + Vite + TypeScript UI. Contains routes, pages, Zustand auth/UI stores, Axios API helpers, and SSE streaming helpers.
- backend/: FastAPI application. backend/main.py creates the app, mounts routers, configures middleware, and initializes RAG and the knowledge graph.
- backend/api/v1/: API routers for auth, tutor, agent, assessment, mentor, evaluation, revision, visualization, and shared feature routers.
- synapse_ai_tutor/backend/: framework-agnostic AI helpers for RAG, knowledge graph, memory, progress, LLM, TTS, STT, notes, roadmap, and assessment.
- synapse_ai_tutor/ai/: newer modular AI code for RAG, LLM routing, and evaluation.
- synapse_ai_tutor/storage/: JSON repositories, SQLAlchemy models, PostgreSQL repositories, and migrations.
- synapse_ai_tutor/data/: persistent JSON data, FAISS index, chunks, knowledge graph, users, progress, student memory, goals, and books.

3. Frontend Flow

The frontend entry point is frontend/src/App.tsx. It defines lazy-loaded pages and protected routes. The login page is public, while dashboard, tutor, assessment, graph, notes, roadmap, profile, concepts, visualize, and study pages are protected.

- frontend/src/lib/api.ts contains typed Axios API helpers.
- frontend/src/lib/sse.ts handles server-sent event streaming for tutor responses.

- frontend/src/store/authStore.ts stores user state and manages login, logout, token refresh, and hydration.
- frontend/src/features/tutor/TutorPage.tsx sends user questions and renders streamed tutor responses.

Authentication tokens are saved in localStorage under synapse_access_token and synapse_refresh_token. Axios attaches the access token to protected requests. If a request returns 401, the interceptor attempts refresh and retries.

4. Backend Startup and Routing

backend/main.py creates the FastAPI app. During lifespan startup it loads environment variables, initializes the RAG pipeline unless SKIP_RAG_INIT is truthy, loads the NetworkX knowledge graph, and stores shared objects on app.state.

- app.state.rag_pipeline stores the RAGPipeline singleton.
- app.state.knowledge_graph stores the NetworkX knowledge graph.
- backend/dependencies.py provides get_username, get_rag_pipeline, and get_knowledge_graph.
- backend/api/v1/router.py aggregates all routers under /api/v1.

Most routers depend on get_username, which decodes the JWT bearer token and extracts the subject username.

5. Authentication

- Files: backend/api/v1/auth.py, backend/schemas/auth.py, synapse_ai_tutor/data/users.json, synapse_ai_tutor/data/refresh_tokens.json.
- Responsibilities: user registration, login, bcrypt password verification, JWT access token issue, refresh token issue, logout, and /me profile lookup.

```
Login form -> POST /api/v1/auth/login
           -> users.json lookup
           -> bcrypt password verification
           -> JWT access + refresh token
           -> frontend localStorage
           -> protected API calls
```

Demo credentials are demo/demo123, admin/admin123, and student/student123. The MVP stores users in users.json and refresh tokens in refresh_tokens.json.

6. RAG Pipeline

- Files: synapse_ai_tutor/backend/rag.py, chunking.py, embeddings.py, retriever.py, backend/api/v1/routers.py RAG router, data/books, data/faiss_index.bin, chunks.pkl or chunks.json.

The RAG pipeline retrieves relevant textbook context for tutor responses. It processes PDFs from the books folder, extracts pages, chunks text, generates embeddings, builds a FAISS index, and returns top-k relevant chunks for a user query.

- Embedding model: BAAI/bge-large-en-v1.5, with fallback to sentence-transformers/all-MiniLM-L6-v2.
- Vector search: FAISS IndexFlatIP with normalized vectors, which approximates cosine similarity.
- Retrieval: the student question is embedded and searched against the FAISS index. Returned chunks include text, source, page, and relevance score.

```
PDF books -> text extraction -> chunks -> embeddings -> FAISS index
Student question -> query embedding -> FAISS top-k chunks
Chunks -> LLM prompt reference material -> answer + sources
```

The RAG upload endpoint accepts PDFs, saves them under data/user_docs/<username>/, chunks them, rebuilds the FAISS index, and refreshes the live RAG pipeline.

Limitations: RAG init can be slow, retrieval quality depends on a healthy index and matching embedding model, and the codebase still has some legacy chunks.pkl versus chunks.json format history.

7. Knowledge Graph and GraphRAG

- Files: synapse_ai_tutor/backend/knowledge_graph.py, graph_rag.py, data/knowledge_graph.json, backend/api/v1/routers.py graph router, frontend/src/features/graph/KnowledgeGraphPage.tsx.

The knowledge graph is loaded from JSON into a NetworkX directed graph. It contains topic nodes, concept nodes, and relation edges. The frontend visualizes it using D3 force simulation.

- expand_query(question, topic) finds graph concepts in the question and expands the query with neighbors and prerequisites.
- graph_learning_path(concept, topic) computes a path from topic to concept using shortest path logic.
- graph_rag_retrieve() expands a query through the graph, runs FAISS retrieval, and reranks chunks by graph concept overlap.

```
Question + topic -> concept match -> graph neighbors
                 -> expanded query -> FAISS retrieval
                 -> concept-overlap reranking -> chunks for prompt
```

Limitations: the graph is static, concept matching is string-based, and reranking is lightweight rather than deep graph reasoning.

8. Student Memory

- Files: synapse_ai_tutor/backend/student_memory.py, synapse_ai_tutor/data/student_memory.json, memory/chat endpoints in backend/api/v1/routers.py.

Student memory is the digital twin layer. It stores short-term conversation history, quiz history, learning events, mistakes, recent topics, and derived student summaries.

- add_message() appends user/assistant messages per topic and keeps a rolling 20-message window.
- get_recent_messages() returns recent conversation context.
- add_quiz_result() stores assessment/quiz outcomes and mistakes.
- add_learning_event() stores behavior events such as tutor sessions and quiz completions.
- generate_student_summary() produces weak topics, strong topics, recent mistakes, recent topics, streak, quiz count, and learning style.

This memory is reused in tutor prompts, dashboard analytics, mentor brief generation, and personalization logic.

9. Progress Tracking

- Files: synapse_ai_tutor/backend/progress_tracker.py, synapse_ai_tutor/data/progress.json.

Progress tracking stores topic-level mastery, level, score, max score, knowledge gaps, assessment history, practice history, sessions, last accessed time, and completion status.

- update_assessment_score() persists assessment outcomes and computes mastery percentage.
- update_mastery_from_practice() adjusts mastery after practice performance.
- get_weak_topics() returns topics with low mastery.
- get_strengths() returns topics with high mastery.
- get_overall_stats() computes dashboard-level summary data.

The tutor uses this data to adapt explanation depth and repair weak areas. The dashboard uses it for mastery maps, trends, recommendations, and recent activity.

10. Assessment and Student Evaluation

- Files: backend/api/v1/assessment.py, synapse_ai_tutor/backend/assessment.py, data/manus-dataset.jsonl, data/assessment_sessions.json.

Assessments are 15-question topic quizzes with 5 easy, 5 intermediate, and 5 hard questions. Difficulty is assigned heuristically from content length and technical keywords. Scoring is weighted: easy = 1, intermediate = 2, hard = 3, max score = 30.

```
GET /assessment/start/{topic}
-> load JSONL dataset
-> categorize questions by topic keyword matching
-> assign difficulty
-> generate MCQs
-> save assessment session

POST /assessment/submit
-> compare selected options with correct_index
-> weighted scoring
-> derive level, mastery, mistakes, gaps
-> update progress.json and student_memory.json
-> return results to frontend
```

Limitations: MCQ distractors are generic and sometimes too easy. Knowledge gaps are coarse and not yet true concept-level diagnosis.

11. Adaptive Tutor Response Generation

- Files: backend/api/v1/tutor.py, backend/api/v1/agent.py, synapse_ai_tutor/backend/llm_client.py, synapse_ai_tutor/ai/llm/router.py.

The tutor builds responses using topic, level, mastery, knowledge gaps, weak topics, strong topics, recent mistakes, recent conversation, retrieved RAG chunks, graph context, and explain mode.

- /tutor/explain is the structured adaptive tutoring endpoint.
- /agent/tutor is the multi-step agentic SSE endpoint used by the current Tutor page.
- Responses are streamed to the frontend as SSE chunks.
- The prompt asks for exact sections: Explanation, Analogy, Worked Example, and Practice Questions.

```
Assessment -> progress.json -> mastery/gaps
Tutor chat -> student_memory.json -> recent context/mistakes
Profile preferences -> learning style
RAG/KG -> reference context
All signals -> LLM system prompt -> adaptive answer
```

LLM provider logic tries Ollama when enabled, then Groq fallback, with optional NVIDIA NIM for research-grade/deep queries. The LLM router classifies queries as simple, code, math, complex, or deep research.

12. Chat and Conversation Storage

- Files: chat router in backend/api/v1/routers.py and synapse_ai_tutor/backend/student_memory.py.

Chat endpoints support history lookup, streaming message response, and clearing a topic history. Chat uses recent conversation, RAG context, LLM generation, and optional citations. Messages are persisted in student_memory.json.

13. TTS

- Files: synapse_ai_tutor/backend/tts.py, voice router in backend/api/v1/routers.py, frontend/src/features/tutor/TutorPage.tsx.

Text-to-speech is used for voice mode. The frontend first uses browser SpeechSynthesis for low-latency local playback. Backend TTS is a fallback that uses gTTS by default and optional ElevenLabs if configured. Generated MP3 files are cached in audio_cache and served through /audio.

```
Tutor answer -> browser SpeechSynthesis
Fallback -> POST /voice/tts -> tts.py -> MP3 cache -> /audio/<file>.mp3
```

Limitations: browser voices vary by machine, gTTS needs internet, and ElevenLabs requires an API key.

14. STT

- Files: synapse_ai_tutor/backend/stt.py, voice router in backend/api/v1/routers.py, TutorPage.tsx.

Speech-to-text records microphone audio using the browser MediaRecorder API, sends an audio blob to /voice/stt, and transcribes using Groq Whisper if enabled or local Whisper/faster-whisper fallback.

```
Browser microphone -> audio blob -> POST /voice/stt
-> temporary audio file -> Whisper transcription -> { transcript }
```

Limitations: local Whisper dependencies are heavy, and audio decoding depends on ffmpeg/package availability.

15. Database and Storage

The active MVP database is JSON-file storage. Important files include users.json, refresh_tokens.json, student_memory.json, progress.json, assessment_sessions.json, knowledge_graph.json, goals.json, revision_schedule.json, feedback.json, chunks, and faiss_index.bin.

The SQLAlchemy layer in synapse_ai_tutor/storage/models.py defines future PostgreSQL tables: users, assessments, knowledge_state, learning_sessions, chat_messages, quiz_results, roadmaps, notes, bookmarks, goals, user_preferences, student_profile, concept_mastery, and revision_schedule.

The storage pattern uses atomic JSON writes with temporary files and os.replace, plus simple repository classes in storage/json_store.py. PostgreSQL repositories exist but are not the active default runtime path.

16. Dashboard, Mentor, and Revision

Dashboard endpoints compute total sessions, streak, topics studied, average mastery, strongest/weakest topics, recent activity, mastery trends, weekly activity, recommendations, and goals.

The mentor router creates a personalized daily brief using progress, weak topics, strong topics, recent topics, streaks, and active goals. Feedback is stored as thumbs-up/down ratings in feedback.json.

The revision router implements SM-2 spaced repetition. It schedules concepts, returns due reviews, and updates intervals based on quality scores from 0 to 5.

17. Evaluation

- Files: backend/api/v1/evaluation.py and synapse_ai_tutor/ai/evaluation/evaluator.py.

The system includes lightweight RAGAS-style metrics: groundedness, faithfulness, retrieval precision, and overall quality. These are based on keyword overlap between query, answer, and retrieved context chunks. Feedback stats are derived from thumbs-up/down user ratings.

Limitations: this is a lightweight heuristic evaluator, not full RAGAS or a human-validated evaluation pipeline.

18. Design Patterns and Algorithms

- FastAPI dependency injection for auth, RAG, and knowledge graph access.
- Singleton app state for expensive RAG and KG resources.
- Repository pattern and atomic JSON persistence.
- Lazy-loaded React pages and protected routes.
- SSE streaming for real-time tutor output.
- JWT access and refresh token authentication.
- FAISS dense retrieval with normalized embeddings.
- NetworkX graph traversal and query expansion.
- BM25 and Reciprocal Rank Fusion in the newer hybrid retriever module.
- SM-2 spaced repetition scheduling.
- Heuristic assessment difficulty assignment and weighted scoring.
- Rule-based query classification for model routing.

19. Current Limitations

- PostgreSQL schema exists, but runtime still primarily uses JSON storage.
- RAG cache has legacy format history and can be slow to initialize.
- Assessment questions and distractors are heuristic and need quality improvement.
- Knowledge graph is static and not automatically generated from books or student behavior.
- Evaluation metrics are lightweight keyword-overlap approximations.
- Voice features depend on browser support, local dependencies, or external APIs.
- Some APIs and modules show MVP-stage wiring rather than production-grade integration.

20. Review-Ready Presentation Summary

Synapse AI Tutor is an adaptive learning platform built with React and FastAPI. The frontend provides protected learning pages for tutoring, assessment, dashboard, knowledge graph, notes, roadmap, voice, and study sessions. The backend exposes all features under /api/v1 and uses JWT authentication. The MVP persists data in JSON files for simplicity and durability, while SQLAlchemy models define the future PostgreSQL schema.

The AI tutor adapts responses using four main signals: RAG textbook retrieval, knowledge graph expansion, persistent student memory, and progress tracking. When a student asks a question, the system retrieves relevant textbook chunks using FAISS, optionally expands the query through a NetworkX knowledge graph, loads the student?s mastery, gaps, weak topics, recent mistakes, and conversation history, then builds a structured tutor prompt. The LLM response is streamed back over SSE and displayed in sections: Explanation, Analogy, Worked Example, and Practice Questions.

Assessments generate 15 weighted MCQs per topic, score the student, update mastery and level, store knowledge gaps, and feed those results back into future tutor prompts. Student behavior is tracked through conversation history, quiz history, learning events, recent topics, streaks, goals, and spaced repetition schedules. TTS and STT add voice interaction using browser speech, gTTS, ElevenLabs, Groq Whisper, and local Whisper options.

The system is a strong MVP because it connects retrieval, memory, progress, evaluation, and adaptive prompting into one learning loop. Its main limitations are heuristic assessment quality, incomplete PostgreSQL migration, static knowledge graph, lightweight evaluation metrics, and some RAG/voice dependency fragility.